

FRAMMER AI

IMPLEMENTATION PLAN DOCUMENT

Frammer Analytics OS

Full-Stack B2B Analytics Dashboard
for Media Operations Intelligence

Project	Frammer Analytics OS
Version	1.0
Date	June 2026
Stack	FastAPI · React · PostgreSQL · Gemini AI
Architecture	Star Schema · JWT RBAC · ReAct Agent

Frammer AI Pvt Limited | CIN U62099DL2023PTC411035
Registered Office: 207 Okhla Industrial Estate, New Delhi - 110020
corp@frammer.com | +919711204165

Table of Contents

1.	Background & Overview
2.	Tech Stack
3.	Project Structure
4.	Data Model (Star Schema)
5.	Authentication & RBAC
6.	API Endpoints
7.	ATLAS AI Agent Architecture
8.	Data Simulator
9.	Frontend Design System
10.	Docker Deployment
11.	Phased Build Plan
12.	Verification Plan

1) Background & Overview

Frammer AI is a B2B platform used by media teams to convert long-form video and live content into short-form vertical videos, chapters, summaries, and other publish-ready outputs with metadata — with just one click.

Frammer already has an internal analytics dashboard used to track:

- **Uploaded / Processed / Published videos** (count and hours)
- **Duration-based usage**
- **Channel-wise performance**
- **Video category/type breakdowns** (reels, shorts, chapters, summaries, viral videos)
- **Video-level details** (headline, source, team, uploader, publish status, platform)

As the number of clients, channels/workspaces, users, languages, and output types grows, Frammer now wants a more **scalable, user-friendly, and insight-ready dashboard** that is easier to navigate and can support future feature additions.

What We're Building

Frammer Analytics OS — a full-stack analytics platform that:

- Improves the current reporting experience for internal and client-facing use
- Supports **multi-dimensional drill-downs** and clear KPI tracking
- Is **easy to navigate**, even for non-technical users
- Has a **clean, modular structure** so new metrics/dimensions can be added later
- Surfaces **actionable insights**, not just raw counts and charts
- Includes an **AI-powered natural language query agent (ATLAS)** for conversational analytics

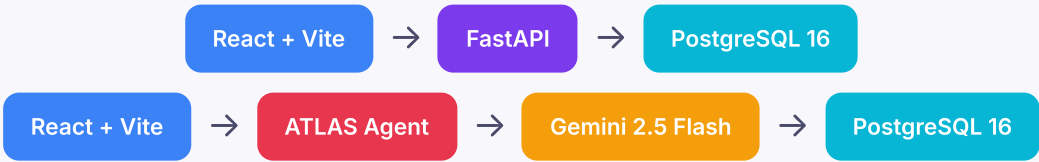
⚡ IMPORTANT CONTEXT

A single client may have **multiple channels/workspaces**, and each channel/workspace may have **different users, languages, and usage patterns**. The dashboard handles this structure with role-based data scoping.

2) Tech Stack

LAYER	TECHNOLOGY	PURPOSE
Backend	Python 3.12, FastAPI	REST API server with async support
ORM	SQLAlchemy 2.0	Database models, query building
Database	PostgreSQL 16	Star schema data warehouse
Auth	PyJWT, bcrypt	JWT tokens, password hashing
Data Processing	pandas	Data transformation, aggregation
AI Agent	Google Gemini 2.5 Flash	Natural language analytics (ATLAS)
AI SDK	google-genai, FastMCP	LLM client, tool registry
Frontend	React 18 + Vite	SPA with fast HMR
UI Components	shadcn/ui + Tailwind CSS	Premium component library
Charts	Recharts	Responsive data visualizations
State	Zustand	Lightweight global state management
Deployment	Docker + docker-compose	3-service containerized deployment

System Architecture Overview



3) Project Structure

```

frammer-analytics/
├── docker-compose.yml
├── .env.example
├── README.md
├── api/                                # FastAPI backend
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── alembic.ini
│   ├── alembic/
│   │   ├── env.py
│   │   └── versions/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py                    # FastAPI app + CORS + lifespan
│   │   ├── config.py                 # Settings via pydantic-settings
│   │   ├── database.py               # SQLAlchemy engine + session
│   │   ├── models/                  # SQLAlchemy ORM models
│   │   │   ├── __init__.py
│   │   │   ├── fact_videos.py
│   │   │   ├── dim_client.py
│   │   │   ├── dim_channel.py
│   │   │   ├── dim_user.py
│   │   │   ├── dim_type.py
│   │   │   └── dim_platform.py
│   │   ├── schemas/                 # Pydantic request/response schemas
│   │   │   ├── __init__.py
│   │   │   ├── auth.py
│   │   │   ├── dashboard.py
│   │   │   ├── filters.py
│   │   │   └── agent.py
│   │   ├── routes/                  # API route modules
│   │   │   ├── auth.py              # POST /login, /register, /me
│   │   │   ├── executive.py         # GET /api/executive/...
│   │   │   ├── trends.py            # GET /api/trends/...
│   │   │   ├── analysis.py          # GET /api/analysis/...
│   │   │   ├── funnel.py            # GET /api/funnel/...
│   │   │   ├── explorer.py          # GET /api/explorer/...
│   │   │   ├── filters.py           # GET /api/filters/options
│   │   │   └── agent.py             # POST /api/agent/query
│   │   ├── services/                # Business logic
│   │   │   ├── auth_service.py
│   │   │   ├── dashboard_service.py
│   │   │   ├── query_builder.py     # CTE-based tenant scoping
│   │   │   └── data_quality.py
│   │   ├── agent/                   # ATLAS AI agent
│   │   └── engine.py                # ReAct reasoning loop

```

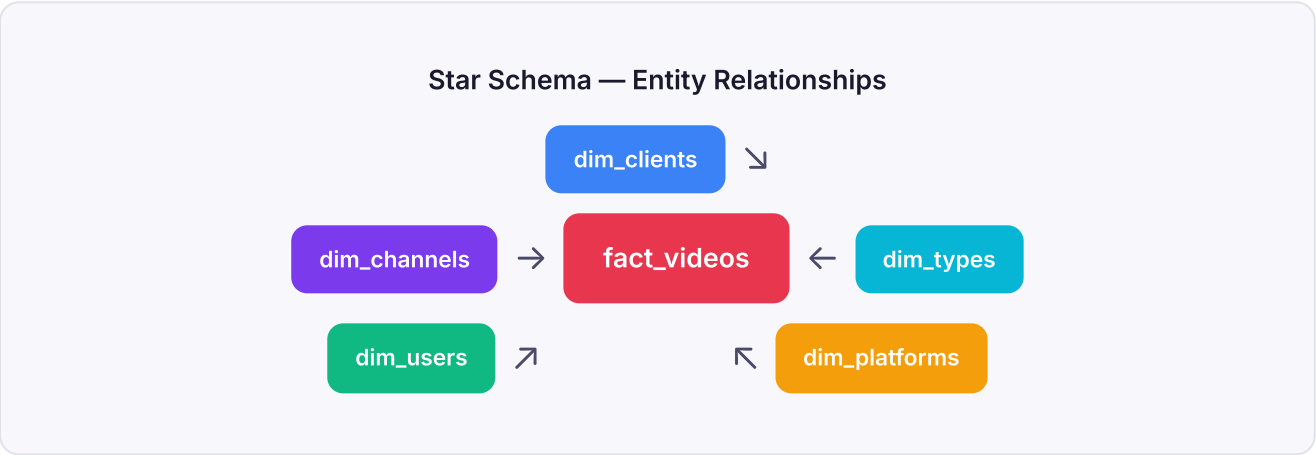
```

├── tools.py           # Tool definitions
├── intent.py         # Intent classifier
├── prompts.py        # System prompts
├── schema_context.py # Schema for LLM
├── middleware/
│   └── rbac.py        # JWT + role-based CTE injection
├── scripts/
│   ├── seed_schema.py
│   └── simulate_data.py # 6 months synthetic data
├── web/               # React + Vite frontend
│   ├── Dockerfile
│   ├── package.json
│   ├── vite.config.js
│   ├── index.html
│   └── src/
│       ├── main.jsx
│       ├── App.jsx
│       ├── index.css
│       ├── components/
│       │   ├── ui/           # shadcn/ui primitives
│       │   ├── layout/       # Sidebar, Header, DashboardLayout
│       │   ├── charts/       # KPICard, TrendChart, FunnelChart...
│       │   ├── filters/      # GlobalFilterBar, DateRangePicker...
│       │   └── agent/        # AtlasChat, ChatMessage, AgentChart
│       ├── pages/            # 5 dashboard pages + login
│       ├── stores/           # Zustand stores
│       ├── hooks/            # Custom React hooks
│       └── lib/              # API client, utils, constants
├── docs/
│   ├── metric_dictionary.md
│   ├── dimension_dictionary.md
│   └── data_model.md

```

4) Data Model (Star Schema)

The data model follows a **star schema** pattern optimized for analytical queries — one central fact table surrounded by dimension tables for clean, modular drill-downs.



Dimension Tables

dim_clients

COLUMN	TYPE	CONSTRAINT	NOTES
client_id	SERIAL	PK	Auto-increment primary key
client_name	VARCHAR(255)	NOT NULL	e.g., "Times Network"
client_segment	VARCHAR(100)		"Enterprise", "SMB", "Startup"
region	VARCHAR(100)		"North India", "South India", etc.
created_at	TIMESTAMP	DEFAULT NOW()	

dim_channels

COLUMN	TYPE	CONSTRAINT	NOTES
channel_id	SERIAL	PK	
client_id	INTEGER	FK → dim_clients	Parent client
channel_name	VARCHAR(255)	NOT NULL	e.g., "Times Now"

COLUMN	TYPE	CONSTRAINT	NOTES
workspace	VARCHAR(255)		Workspace identifier
language	VARCHAR(50)		"Hindi", "English", "Tamil", etc.
created_at	TIMESTAMP	DEFAULT NOW()	

dim_users

COLUMN	TYPE	CONSTRAINT	NOTES
user_id	SERIAL	PK	
client_id	INTEGER	FK → dim_clients	Belongs to client
username	VARCHAR(255)	NOT NULL	
email	VARCHAR(255)	UNIQUE	Login credential
team_name	VARCHAR(255)		"Editorial", "Production", etc.
role	VARCHAR(50)	NOT NULL	website_admin / client_admin / user
password_hash	VARCHAR(255)	NOT NULL	bcrypt hashed
created_at	TIMESTAMP	DEFAULT NOW()	

dim_types

COLUMN	TYPE	CONSTRAINT	NOTES
type_id	SERIAL	PK	
input_type	VARCHAR(100)		Speech, Interview, Special Report, MKM, Live
output_type	VARCHAR(100)		Reels, Chapters, Shorts, Summary, Viral

dim_platforms

COLUMN	TYPE	CONSTRAINT	NOTES
platform_id	SERIAL	PK	
platform_name	VARCHAR(100)	NOT NULL	YouTube, Instagram, Twitter, Facebook, LinkedIn

Fact Table

fact_videos

COLUMN	TYPE	CONSTRAINT	NOTES
video_id	VARCHAR(50)	PK	UUID or job_id
client_id	INTEGER	FK → dim_clients	
channel_id	INTEGER	FK → dim_channels	
user_id	INTEGER	FK → dim_users	
type_id	INTEGER	FK → dim_types	
platform_id	INTEGER	FK → dim_platforms	NULL if not published
headline	VARCHAR(500)		Video headline/title
source_url	TEXT		Original source URL
uploaded_at	TIMESTAMP	NOT NULL	Upload timestamp
processed_at	TIMESTAMP		NULL if not processed
published_at	TIMESTAMP		NULL if not published
published_flag	BOOLEAN	DEFAULT false	
duration	INTEGER		Duration in seconds
published_url	TEXT		Published URL (if published)
billable_flag	BOOLEAN	DEFAULT true	

Indexes

```
CREATE INDEX idx_fact_uploaded_at ON fact_videos(uploaded_at);
CREATE INDEX idx_fact_client_id ON fact_videos(client_id);
CREATE INDEX idx_fact_channel_id ON fact_videos(channel_id);
CREATE INDEX idx_fact_published ON fact_videos(published_flag);
CREATE INDEX idx_fact_type_id ON fact_videos(type_id);
CREATE INDEX idx_fact_composite ON fact_videos(client_id, channel_id, uploaded_at);
```

5) Authentication & RBAC

JWT Authentication Flow



Every authenticated request includes `Authorization: Bearer <token>`. The JWT payload contains:

```
{
  "user_id": 42,
  "client_id": 5,
  "role": "client_admin",
  "exp": 1720000000
}
```

Three Roles

ROLE	SCOPE	USE CASE
website_admin	Sees ALL data across all clients	Frammer internal team
client_admin	Sees data for their own client only	Client leadership / managers
user	Sees only their own uploaded data	Individual editors / producers

CTE-Based Query Scoping

Every database query is automatically wrapped with a Common Table Expression (CTE) that filters data based on the authenticated user's role. This ensures **zero data leakage** across tenants.

```
# query_builder.py – Role-based CTE injection
def build_scoped_query(base_query: str, user: dict) -> str:
    if user["role"] == "website_admin":
        return base_query # No filter – sees everything

    elif user["role"] == "client_admin":
        return f"""
```

```

WITH scoped_videos AS (
    SELECT * FROM fact_videos
    WHERE client_id = %(client_id)s
)
{base_query.replace('fact_videos', 'scoped_videos')}
"""

else: # role == "user"
    return f"""
WITH scoped_videos AS (
    SELECT * FROM fact_videos
    WHERE user_id = %(user_id)s
)
{base_query.replace('fact_videos', 'scoped_videos')}
"""

```

⚠️ SECURITY NOTE

All queries use parameterized inputs (not string interpolation) to prevent SQL injection. The CTE approach above is simplified for documentation clarity.

6) API Endpoints

A) Authentication

METHOD	ENDPOINT	DESCRIPTION
POST	/api/auth/register	Register new user
POST	/api/auth/login	Login — returns JWT token
GET	/api/auth/me	Get current authenticated user profile

B) Global Filters

METHOD	ENDPOINT	DESCRIPTION
GET	/api/filters/options	All filter options (clients, channels, users, types, platforms)

C) Executive Summary

METHOD	ENDPOINT	DESCRIPTION
GET	/api/executive/kpis	KPI cards — uploaded/processed/published counts + durations
GET	/api/executive/sparklines	30-day trend sparklines for each KPI
GET	/api/executive/alerts	Anomaly alerts (drop-offs, usage spikes)

D) Usage & Trends

METHOD	ENDPOINT	DESCRIPTION
GET	/api/trends/timeseries	Time series data with granularity toggle (daily/weekly/monthly)
GET	/api/trends/comparison	Period-over-period comparison data

E) Client / Channel / User Analysis

METHOD	ENDPOINT	DESCRIPTION
GET	/api/analysis/pivot	Double-dimension pivot (dim1 × dim2)
GET	/api/analysis/leaderboard	Top N entities by selected metric
GET	/api/analysis/drilldown	Drill into a specific dimension value

F) Publishing Funnel & Type Mix

METHOD	ENDPOINT	DESCRIPTION
GET	/api/funnel/stages	Uploaded → Processed → Published counts
GET	/api/funnel/conversion	Publish rate % with breakdowns
GET	/api/funnel/type-mix	Output type and input type breakdown

G) Video Explorer

METHOD	ENDPOINT	DESCRIPTION
GET	/api/explorer/videos	Paginated, filterable, searchable video list
GET	/api/explorer/export	CSV export of filtered results

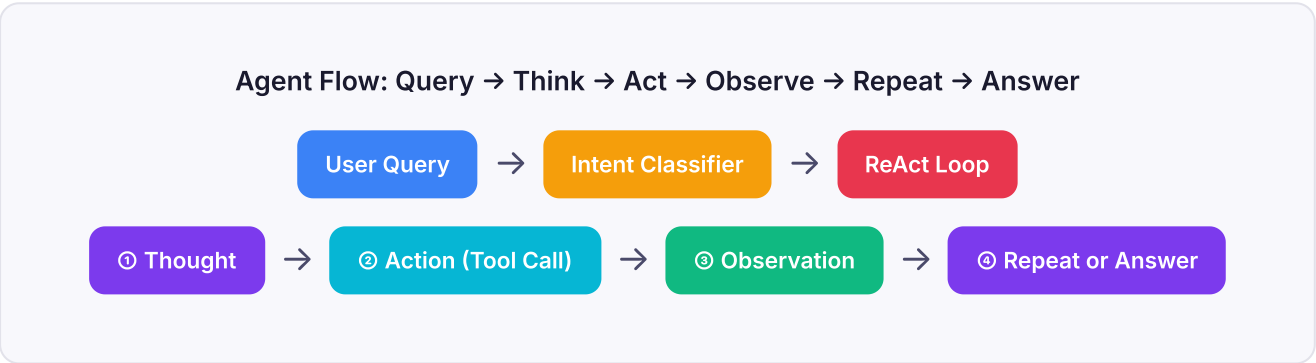
H) ATLAS AI Agent

METHOD	ENDPOINT	DESCRIPTION
POST	/api/agent/query	Natural language query → ATLAS agent response

7) ATLAS AI Agent Architecture

ATLAS (Analytical Thought & Language Agent System) is an AI-powered natural language query engine that allows non-technical users to ask questions about their data in plain English.

ReAct Reasoning Loop



Tool Registry

TOOL	DESCRIPTION	SAFETY
<code>execute_query</code>	Runs a read-only SQL SELECT against PostgreSQL	CTE scoping + SELECT-only regex + 10s timeout
<code>build_chart</code>	Returns chart config JSON (type, data, labels)	Schema-validated output
<code>get_schema</code>	Returns the star schema DDL + relationships	Read-only
<code>get_metric_definitions</code>	Returns metric dictionary with definitions + formulas	Read-only
<code>answer</code>	Final answer — terminates the loop	Max 5 iterations

Example Queries ATLAS Can Handle

- "Show channel-wise usage hours for interviews vs speeches last month"
- "Which channels have the biggest processed vs published gap?"
- "Top 10 videos by duration and their publish status"

- "What is the publish conversion % by channel / type / user?"
- "How much data is missing or marked as Unknown?"

Safety Guardrails

- All queries wrapped in CTE scoping per user role
- Only **SELECT** statements allowed (regex validation)
- Query timeout: 10 seconds
- Maximum ReAct iterations: 5
- Intent classifier routes conversational queries away from the agent

8) Data Simulator

The `simulate_data.py` script generates **6 months** of realistic synthetic data to populate the dashboard with meaningful patterns.

Data Volume

ENTITY	COUNT	GENERATION LOGIC
Clients	8	Mix of Enterprise / SMB / Startup across regions
Channels	~30	2–6 per client, various languages (Hindi, English, Tamil, etc.)
Users	~50	3–10 per client, assigned to teams
Types	~30	~6 input types × ~5 output types
Platforms	5	YouTube, Instagram, Twitter, Facebook, LinkedIn
Videos (Fact)	~15,000	Realistic distribution over 180 days

Realism Patterns

- **Weekday bias:** Weekday volume ~3× higher than weekends
- **Processing rate:** ~85% of uploaded videos get processed (avg 2-hour delay)
- **Publishing rate:** ~60% of processed videos get published (avg 6-hour delay)
- **Enterprise volume:** Enterprise clients have 3–5× higher upload volume than SMBs
- **Channel variance:** Some channels are significantly more active than others
- **Duration distribution:** Log-normal distribution (30s–600s), mimicking real short-form video lengths
- **Trend injection:** Gradual growth trend + seasonal spikes (election weeks, IPL, etc.)
- **Data quality issues:** ~5% of records have missing team names, unknown platforms, or null URLs

9) Frontend Design System

Visual Design Principles

- **Dark-mode primary:** Slate-900 backgrounds with vibrant accent gradients
- **Typography:** Inter font family (Google Fonts) — clean, modern, highly legible
- **Glassmorphism:** Cards with subtle `backdrop-blur`, semi-transparent borders
- **Micro-animations:** Framer Motion for page transitions, card entrances, hover effects
- **Charts:** Recharts with custom gradient fills, animated tooltips, and responsive sizing

Layout Structure

COMPONENT	BEHAVIOR
Sidebar	Fixed left, collapsible, icons + labels for 5 tabs + ATLAS chat toggle
Header	Top bar — global filters (date range, client, channel) + user avatar + role badge
Content	Scrollable main area with responsive CSS Grid layout
ATLAS Panel	Slide-out right panel for AI chat — overlays content

Dashboard Pages (5 Tabs)

Tab 1: Executive Summary

- 6 KPI cards (uploaded/processed/published × count + duration) with period comparison
- 30-day sparkline trends for each KPI
- Anomaly alert cards (significant drop-offs or spikes)
- Quick-glance donut charts for output type distribution

Tab 2: Usage & Trends

- Full-width time series area chart (daily/weekly/monthly toggle)
- Period-over-period comparison overlay (current vs previous period)
- Channel-wise uploaded/processed/published comparison bar chart
- Channel-wise summary table

Tab 3: Client / Channel / User Analysis

- Dimension selector (dim1 × dim2) for multi-dimensional pivot
- Interactive pivot table with heatmap coloring
- Leaderboard chart (top 10 by selected metric)
- Drill-down panel — click any dimension value to see detailed breakdown

Tab 4: Publishing Funnel & Type Mix

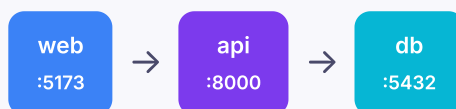
- Visual funnel: Uploaded → Processed → Published (with drop-off %)
- Conversion rate cards with trend arrows
- Output type breakdown (horizontal stacked bar)
- Input type breakdown (treemap or donut)

Tab 5: Video Explorer

- Full-width searchable data table with sortable columns
- Columns: Headline, Source, Published, Team, Type, Uploaded By, Video ID, Platform, URL
- Advanced filters (type, status, date range, channel)
- Pagination (25/50/100 per page)
- CSV export button

10) Docker Deployment

Service Architecture



docker-compose.yml — 3 Services

SERVICE	IMAGE	PORT	NOTES
db	postgres:16-alpine	5432	Volume-persisted data, health check via pg_isready
api	Custom (Python 3.12-slim)	8000	Depends on db, auto-runs migrations + seed
web	Custom (Node 20-alpine)	5173	Vite dev server, proxies /api → api:8000

Environment Variables

```
# .env.example
POSTGRES_DB=frammer_analytics
POSTGRES_USER=frammer
POSTGRES_PASSWORD=frammer_secret
DATABASE_URL=postgresql://frammer:frammer_secret@db:5432/frammer_analytics
JWT_SECRET=your-secret-key-change-in-production
JWT_ALGORITHM=HS256
JWT_EXPIRY_HOURS=24
GEMINI_API_KEY=your-google-ai-studio-api-key
VITE_API_URL=http://localhost:8000
```

11) Phased Build Plan

1 Scaffolding & Infrastructure

- Create full folder structure (api/ + web/ + docs/)
- Write `docker-compose.yml`, `Dockerfile.api`, `Dockerfile.web`
- FastAPI `main.py` with CORS, lifespan hook, health check
- Stub 5 route modules with placeholder endpoints
- PostgreSQL connection via SQLAlchemy
- `.env.example` with all configuration variables

2 Data Model & Simulator

- SQLAlchemy ORM models for all dimension + fact tables
- Alembic migration setup + initial migration
- `simulate_data.py` — generate 6 months of realistic synthetic data
- Verify data with sample SQL queries

3 Auth & RBAC

- User model with bcrypt password hashing
- JWT token generation + validation
- Login / Register / Me endpoints
- RBAC middleware with CTE query scoping
- 3 test users (one per role: admin, client_admin, user)

4 Dashboard API Endpoints

- Query builder service with CTE scoping
- Executive summary endpoints (KPIs, sparklines, alerts)
- Usage & trends endpoints (timeseries, comparison)
- Analysis endpoints (pivot, leaderboard, drilldown)
- Funnel endpoints (stages, conversion, type-mix)
- Explorer endpoints (paginated list, CSV export)
- Global filter options endpoint

5 Frontend Foundation

- Vite + React project setup with Tailwind CSS
- shadcn/ui component installation
- Design system (CSS tokens, dark theme, typography)
- Layout components (Sidebar, Header, DashboardLayout)
- Zustand stores (auth, filters, agent)
- Axios API client with JWT interceptor
- Login page + React Router with protected routes

6 Dashboard Pages

- Executive Summary — KPI cards, sparklines, anomaly alerts
- Usage & Trends — time series charts, period comparison, granularity toggle
- Client/Channel/User Analysis — pivot table, leaderboard, drill-down
- Publishing Funnel — funnel chart, conversion rates, type mix breakdown
- Video Explorer — data table, search, filters, pagination, CSV export

7 ATLAS AI Agent

- Gemini client setup with google-genai SDK
- Intent classifier (conversational vs analytics)
- ReAct reasoning loop engine
- Tool implementations (execute_query, build_chart, get_schema, get_metric_definitions, answer)
- Agent API endpoint
- Frontend chat UI (AtlasChat slide-out panel)
- Dynamic chart rendering from agent responses

8 Polish & Documentation

- Data quality monitoring views
- Error handling + loading states + skeleton screens
- Responsive design for tablet/mobile
- Metric dictionary + dimension dictionary docs
- README with full setup instructions

12) Verification Plan

Automated Checks

```
# 1. Health check
curl http://localhost:8000/health

# 2. Verify data population
docker-compose exec db psql -U frammer -d frammer_analytics \
  -c "SELECT COUNT(*) FROM fact_videos;"

# 3. Test auth flow
curl -X POST http://localhost:8000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "admin@frammer.com", "password": "admin123"}'

# 4. Test KPI endpoint with token
curl -H "Authorization: Bearer <token>" \
  http://localhost:8000/api/executive/kpis

# 5. Test ATLAS agent
curl -X POST http://localhost:8000/api/agent/query \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d '{"query": "Show top 5 channels by publish rate"}'
```

Manual Verification

#	TEST	EXPECTED OUTCOME
1	Login with each role (admin, client_admin, user)	Role-appropriate data shown; no cross-tenant leakage
2	Navigate all 5 dashboard tabs	All charts render with data, no empty states
3	Apply global filters (date range, client, channel)	All pages update consistently
4	Test double-dimension pivot (Channel × Output Type)	Correct cross-tabulation with totals
5	Export CSV from Video Explorer	Downloads complete filtered dataset

#	TEST	EXPECTED OUTCOME
6	Ask ATLAS: "Which channels have the lowest publish rate?"	Agent reasons, queries DB, returns chart + answer
7	Test responsive layout at 768px / 1024px / 1440px	Layout adapts gracefully

Frammer AI Pvt Limited | CIN U62099DL2023PTC411035
Registered Office: 207 Okhla Industrial Estate, New Delhi - 110020
corp@frammer.com | +919711204165